Where the MIPS–PySR SymPy Conversion Failures Occur

What is searched, what is matched, and why the partial baseline cannot yet be scored cleanly

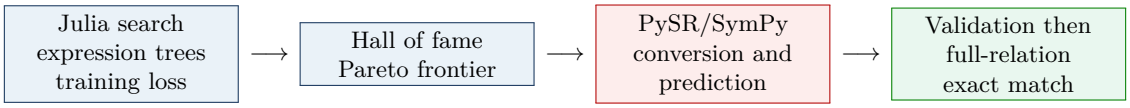
meta_sr reproduction note

26 August 2026

Direct answer. Yes: the visible failures occurred *after* the Julia symbolic-regression search had finished. They occurred while PySR was converting or numerically evaluating its returned expression trees through SymPy. The MIPS "ground-truth match" is not an algebraic comparison to a canonical formula; it is exact behavioral agreement on the complete finite, deduplicated integer transition relation.

1 The short version

The run has two distinct computational worlds:



The search uses the Julia definitions of operators such as `mips_mod`, `mips_floordiv`, and `mips_max`. Once the search returns, Python asks PySR to expose and predict with those expressions. PySR first turns the Julia expression into a SymPy expression and a numerical lambda. That red stage is where the observed failures arise.

The evidence is direct. For failed task index 0, the worker log says:

```
Starting PySR search: 18 train samples, 3 features
PySR search complete ... in 58.2s
PySR Worker finished ... R^2=-1.0000, ERROR: Failed to evaluate the expression
```

The same log prints a Pareto frontier containing a complexity-30 expression with zero training loss. Thus the search produced a candidate; the error was raised afterward. A different seed for the same scalar relation completed search in 60.9 seconds and passed the exact checker.

2 What the 27 scalar problems actually are

For an RNN with hidden state h_t, input u_t, and output o_t, MIPS first learns an integer encoding $z_t = E(h_t) \in \mathbb{Z}^d$. Program extraction is then decomposed into ordinary scalar relations:

$$\text{hidden component } j: \quad x = (z_{t-1,1}, \ldots, z_{t-1,d}, u_t), \quad y = z_{t,j}, \tag{1}$$

$$\text{output component } k: \quad x = (z_{t,1}, \ldots, z_{t,d}), \quad y = o_{t,k}. \tag{2}$$

The raw MIPS traces may contain hundreds of thousands of rows. We deduplicate identical inputs and require a unique target for every input. Only the 27 scalar relations from the ten fully deterministic RNN tasks were sent to PySR. Each relation has a fixed 80/20 train/validation split, but the complete deduplicated table is retained for the final exact check.

This distinction matters: "complete" means every distinct encoded relation row observed in the extracted MIPS data, not every point in an infinite real domain.

3 What "matching" means

Let H be PySR's returned Pareto frontier and let D_{val} and D_{full} be the validation and complete deduplicated relations. For an expression e, define

$$A_D(e) = \frac{1}{|D|} \sum_{(x_i, y_i) \in D} \mathbf{1}\left[\text{finite}(e(x_i)) \ \wedge \ |e(x_i) - y_i| \le 10^{-9}\right].$$

A scalar component is counted as solved exactly when

$$\exists e \in H : \quad A_{D_{\text{val}}}(e) = 1 \quad \wedge \quad A_{D_{\text{full}}}(e) = 1.$$

The representation must also have been diagnosed as deterministic. The checker considers all useful Pareto-front expressions, not only the one PySR's "best" model-selection rule chooses. There is no symbolic simplification or formula-identity requirement in the MIPS domain.

Concrete example: alternating-last4 output coordinate 0

The original trace contains 900,000 rows, but only 11 distinct encoded states. The full exact relation is:

x_0	x_1	y	x_0	x_1	y
0	0	0	8	0	0
0	1	1	31	0	0
1	0	0	32	0	0
6	0	0	37	0	0
$7^\dagger$	0	0	$37^\dagger$	1	1
			38	0	0

The daggered rows are the held-out validation rows. PySR found $e(x_0, x_1) = x_1$ in all ten completed seeds for this component. The check is:

$$e(7, 0) = 0, \qquad e(37, 1) = 1 \quad \text{(validation)},$$

followed by the same equality check on all 11 rows. Because every prediction equals the integer target, this component receives `gt_match_score=1`.

An original RNN task contains multiple scalar components. A whole-RNN solve requires every hidden and output component to be exact; the current evaluator records the components independently, so whole-task results are grouped later.

4 Precisely where a post-search failure can occur

After `model.fit` returns, the worker performs these operations in order:

1. `model.get_best()` selects a row and may force PySR to create SymPy/lambda representations of frontier expressions.
2. The MIPS checker calls `model.predict` for each Pareto expression on validation rows; only validation-perfect expressions are evaluated on the full relation.
3. Independently of the exact checker, the worker calls `model.predict(X_val)` for PySR's selected equation to report R^2 and accuracy.

There are two exception policies:

- Inside the MIPS exact checker, a prediction exception for one frontier equation is caught and the checker moves to the next equation.
- An exception from `get_best()` or from the final unconditional selected-equation prediction escapes to the outer worker handler and is recorded as $R^2 = -1$ with an error.

Therefore the visible worker errors are definitely post-search, but the current logs cannot always distinguish `get_best()` from the final prediction. The generic error is truncated, and the temporary hall-of-fame directory is deleted. Moreover, a conversion failure *inside* the exact checker can be silently converted into a non-match. This is why merely rerunning the rows explicitly labeled "error" is not quite enough for a definitive baseline.

5 Why the current SymPy mappings are fragile

The Julia search operators are protected numerically. For example:

```
mips_mod(x,y) = abs(y) < 1.0e-12 ? 0.0 : mod(x, abs(y))
```

The ternary is lazy: when y is zero, Julia never evaluates the modulo branch. The current SymPy adapter instead constructs

```
Piecewise((0, Eq(y, 0)), (Mod(x, Abs(y)), True))
```

Python evaluates function arguments before calling `Piecewise`. Thus the supposedly protected second branch can be constructed and simplified even when the first branch is true. Directly exercising the adapter reproduces the same error classes seen in the partial run:

Adapter call	Intended protected behavior	Current SymPy result
`mips_mod(x,0)`	return 0	`ZeroDivisionError: Modulo by zero`
`mips_min(nan,x)`	numerical minimum policy	`ValueError: nan is not comparable`
`mips_max(zoo,x)`	numerical maximum policy	`ValueError: zoo is not comparable`
`mips_lt(nan,x)`	protected numerical predicate	`TypeError: Invalid NaN comparison`

Nested expressions make these cases reachable even though the dataset inputs are integers: evolved constants and intermediate division, modulo, minimum, or maximum nodes can create zero denominators, `nan`, or complex infinity (`zoo`) while SymPy simplifies a frontier expression.

There are also two semantic discrepancies that should be tested during the fix:

- Julia treats $|y| < 10^{-12}$ as a zero denominator, whereas the SymPy adapter protects only exact symbolic equality $y = 0$.
- Julia's `mips_xor` rounds both operands before reducing modulo two; the SymPy adapter currently uses $\mathrm{Mod}(x+y, 2)$ without rounding. Intermediate expressions need not be integer-valued.

The domain also defines safer NumPy operator functions, but this PySR prediction path calls `model.predict`; it uses PySR's SymPy/lambda representation, not that separate NumPy namespace.

Important consequence. A reported exact solve is strong evidence because numerical prediction succeeded on the full relation. A reported non-solve is less trustworthy in this run: a promising frontier equation may have been skipped after a conversion exception. The partial solve rate therefore underestimates search performance, and cross-language operator semantics should be checked before treating exact hits as extracted Julia programs.

6 Recommended repair and rerun

The repair should change evaluation, not search:

1. Represent each MIPS operator as an inert custom SymPy function with an explicit numerical implementation, so SymPy does not eagerly rewrite protected branches; or bypass SymPy entirely for MIPS scoring and evaluate expression strings with the existing safe NumPy operator namespace.
2. Use the same faithful evaluator for Pareto validation, full-relation matching, and selected-equation accuracy. Display-oriented symbolic conversion should not be a prerequisite for behavioral scoring.
3. Add cross-backend tests comparing Julia and Python/SymPy behavior on zero and near-zero denominators, non-integer intermediates, `nan`, infinities, and nested operators.
4. Record the failing stage, complete exception, and offending equation; retain the hall of fame on failures. Do not silently turn match-check exceptions into ordinary non-matches.
5. Rerun all 270 baseline fits after the repair. Rerunning only the 48 explicit errors from the 126-result diagnostic snapshot would miss possible false non-matches that were swallowed inside the checker.

The diagnostic snapshot before cancellation contained 126 result files: 58 strict exact hits, 20 valid-looking non-solves, and 48 explicit post-search errors. Those counts are useful for diagnosing the adapter, but they are not a clean baseline comparison.

Local evidence and code locations

- **Search and post-processing order:** `parallel_eval_pysr.py:1403`, `:1420`, `:1424`, `:1441`, and `:1469`.
- **SymPy mappings and exact matcher:** `domains.py:637`, `:692`, and `:726`.
- **Deterministic split construction:** `mips_tasks.py:351`.
- **Failed seed (search completed, then conversion error):** `outputs/mips_pysr_baseline_1h_seed42/slurm_pysr/eval_0001/logs/chunk0_slot_0.out`.
- **Same component, successful exact seed:** `outputs/mips_pysr_baseline_1h_seed42/slurm_pysr/eval_0001/logs/chunk0_slot_5.out`.